COMSATS University Islamabad,
Park Road, Chak Shahzad, Islamabad Pakistan

SOFTWARE REQUIREMENTS SPECIFICATIONS

(SRS Document)

for

Sender-Guided Pathfinding Simulator

version 1.0

By

Muhammad Ashir **CIIT/FA24-BCS-062/ISB**

Muhammad Asad Ali **CIIT/FA24-BCS-061/ISB**

Muhammad Shafy Khan **CIIT/FA24-BCS-083/ISB**

Supervisor

Prof. Farah Saeed

Bachelor of Science in Computer Science (2024-2028)

1. Introduction

This report documents the design and implementation of a Sender-Guided Pathfinding Simulator an interactive desktop application that demonstrates two fundamental AI search algorithms: Uniform Cost Search (UCS) and A* (A-Star) Search. The application operates on a weighted undirected graph representing a simplified city-map layout and allows users to visualize optimal paths across the map in real time.

The unique feature of this simulator is the sender-guided routing model: instead of finding a direct shortest path from a start node to a goal node, the route must pass through an intermediate waypoint called the Sender. This mirrors practical scenarios such as delivery routing, relay-based communication, or multi-stop navigation.

1.1 Objectives

- Implement and compare UCS and A* search on a weighted graph.
- Support a three-point route: Start → Sender → Goal.
- Provide an interactive GUI for map visualization, role assignment, obstacle toggling, and animated path playback.
- Demonstrate admissibility of the Euclidean distance heuristic used in A*.

2. Problem Statement

Given a weighted undirected graph $G = (V, E, w)$ where V is a set of nodes, E is a set of edges, and $w: E \to \mathbb{R}^+$ assigns positive costs to edges, the problem is to find a minimum-cost path from a Start node s to a Goal node g that must pass through a Sender node d (waypoint). Formally:

Minimize: cost(s → d) + cost(d → g)

Subject to:

- The path s → d is a shortest path from s to d.
- The path d → g is a shortest path from d to g.
- Neither sub-path may pass through any node marked as blocked (obstacle).

3. Algorithms Used

3.1 Uniform Cost Search (UCS)

UCS is a complete and optimal graph search algorithm that explores nodes in order of their cumulative path cost from the start node. It is equivalent to Dijkstra's algorithm without an explicit goal-test heuristic. The algorithm expands the node with the lowest g(n) value from the priority queue at each step.

Key Properties

- Complete: Yes — finds a solution if one exists (for finite, non-negative weights).
- Optimal: Yes — always finds the minimum-cost path.
- Time Complexity: $O((V + E) \log V)$ with a binary heap.
- Space Complexity: $O(V)$ for the priority queue and g-score table.
- Heuristic: None ($h(n) = 0$ for all nodes).

3.2 A* Search

A* extends UCS by incorporating a heuristic function h(n) that estimates the remaining cost from node n to the goal. It maintains f(n) = g(n) + h(n) in the priority queue, where g(n) is the actual cost to reach n and h(n) is the heuristic estimate.

Heuristic Used: Euclidean Distance

The heuristic used is scaled Euclidean distance between node pixel positions:

$$h(n) = \sqrt{(x_n - x_goal)^2 + (y_n - y_goal)^2} / 120$$

Dividing by 120 (pixels per unit) keeps h(n) consistently smaller than typical integer edge weights, ensuring admissibility — the heuristic never overestimates the true cost.

Key Properties

- Complete: Yes.
- Optimal: Yes — because h(n) is admissible.
- Time Complexity: O(E) in the best case with a perfect heuristic.
- More efficient than UCS in practice — expands fewer nodes by guiding the search toward the goal.

3.3 Algorithm Comparison

Property	UCS	A*
Heuristic	None (h = 0)	Euclidean / 120
Optimality	Yes	Yes (admissible h)
Completeness	Yes	Yes
Speed	Slower (blind search)	Faster (goal-guided)
Use Case	When heuristic unavailable	When spatial info available

4. System Architecture

The project follows a clean Model-View-Controller (MVC)-inspired architecture, organized into three top-level packages:

4.1 Package Structure

Package / File	Module	Responsibility
core/	node.py	Defines the Node data class (id, x, y)
core/	graph.py	Weighted directed graph with adjacency list, blocked set
core/	default_map.py	Builds the 16-node default city graph
core/	path_manager.py	Orchestrates two-leg routing (Start→Sender→Goal)
algorithms/	ucs.py	Uniform Cost Search implementation
algorithms/	a_star.py	A* Search implementation
algorithms/	heuristics.py	Euclidean heuristic function
gui/	graph_view.py	PyQt5 QGraphicsView — map rendering & animation
gui/	sidebar.py	Control panel — algorithm, mode, run, play, reset
gui/	app.py	Main window, layout assembly, stylesheet
	main.py	Entry point

5. Graph Model

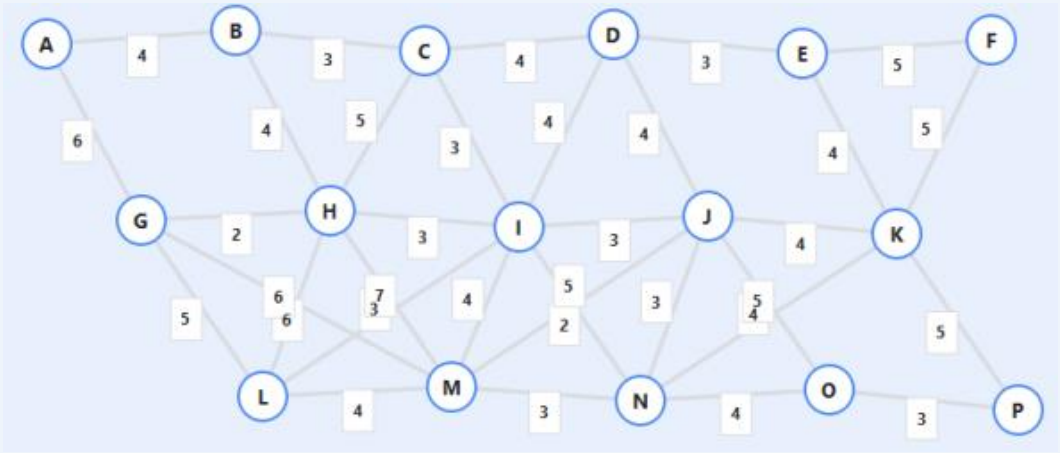
5.1 Default City Map

The default graph consists of 16 nodes (labeled A through P) arranged in three staggered rows simulating a city block layout. Nodes have (x, y) pixel coordinates used both for drawing and computing the Euclidean heuristic.

5.2 Edges and Weights

The graph contains 33 undirected edges. Edge weights represent road travel costs and range from 2 to 7. The adjacency structure is stored as a directed adjacency list (each undirected edge is stored as two directed edges).

5.3 Node Layout



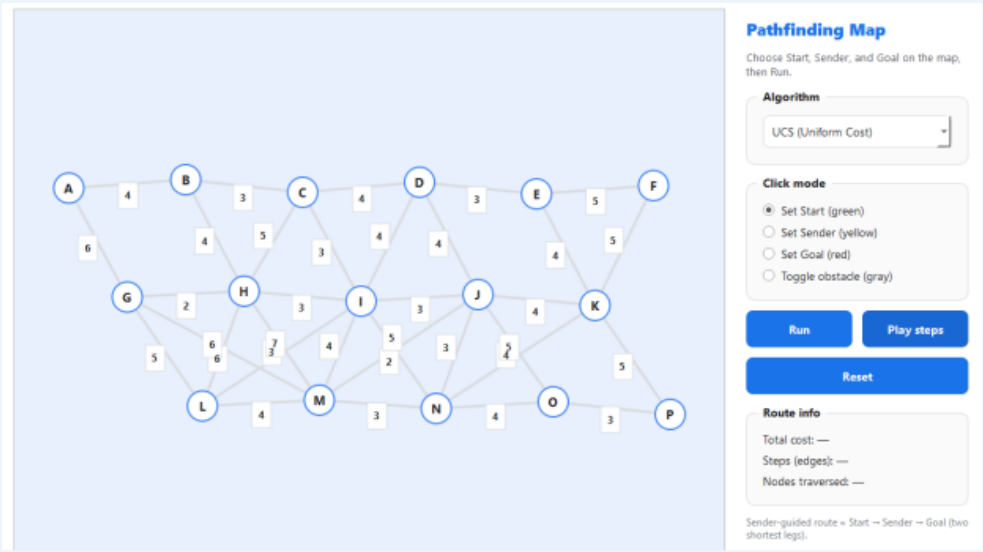
5.4 Blocking / Obstacles

Any node except Start, Sender, and Goal can be toggled as an obstacle (blocked). Blocked nodes are excluded from neighbor expansion during search. The GUI renders blocked nodes in gray (#5f6368).

6. Graphical User Interface

The application uses PyQt5 with a Fusion style and a Google Maps-inspired color scheme. The layout consists of a scrollable map view on the left and a control sidebar on the right.

6.1 Main Window



6.2 Color Coding

Element	Color	Description
Start Node	#34a853 (Green)	User-selected starting point
Goal Node	#ea4335 (Red)	User-selected destination
Sender Node	#fbbc04 (Yellow)	Mandatory waypoint
Blocked Node	#5f6368 (Gray)	Obstacle — excluded from search
Path Highlight	#1a73e8 (Blue)	Optimal route drawn on map
Agent	#9334e6 (Purple)	Animated traveler marker
Default Node	#ffffff / #4285f4	Unassigned node (white fill, blue stroke)

6.3 Click Modes

The sidebar provides four mutually exclusive radio buttons to control what happens when the user clicks a node on the map:

- Set Start — marks the clicked node green as the starting point.
- Set Sender — marks the clicked node yellow as the waypoint.
- Set Goal — marks the clicked node red as the destination.
- Toggle Obstacle — toggles the node between passable and blocked (gray).

7. Implementation Details

7.1 Priority Queue & Tie-Breaking

Both UCS and A* use Python's heapq min-heap. To avoid comparison errors on equal-cost entries with non-comparable node IDs, a monotonically increasing integer counter is inserted as the second element of each heap tuple: (cost, counter, node_id). This guarantees strict FIFO tie-breaking when costs are equal.

7.2 Lazy Deletion

Rather than implementing a decrease-key operation (not natively supported by heapq), both algorithms use lazy deletion: stale entries (where the stored cost exceeds the current best g-score) are silently discarded when popped from the heap. This avoids the overhead of a custom priority queue while maintaining correctness.

7.3 Two-Leg Path Merging

PathManager.find_path() calls the selected search algorithm twice:

- Leg 1: Start → Sender, yielding $path_1$ and $cost_1$.
- Leg 2: Sender → Goal, yielding $path_2$ and $cost_2$.

The merged path is $path_1 + path_2[1:]$ (the Sender node is not duplicated). Total cost is $cost_1 + cost_2$.

7.4 Animation

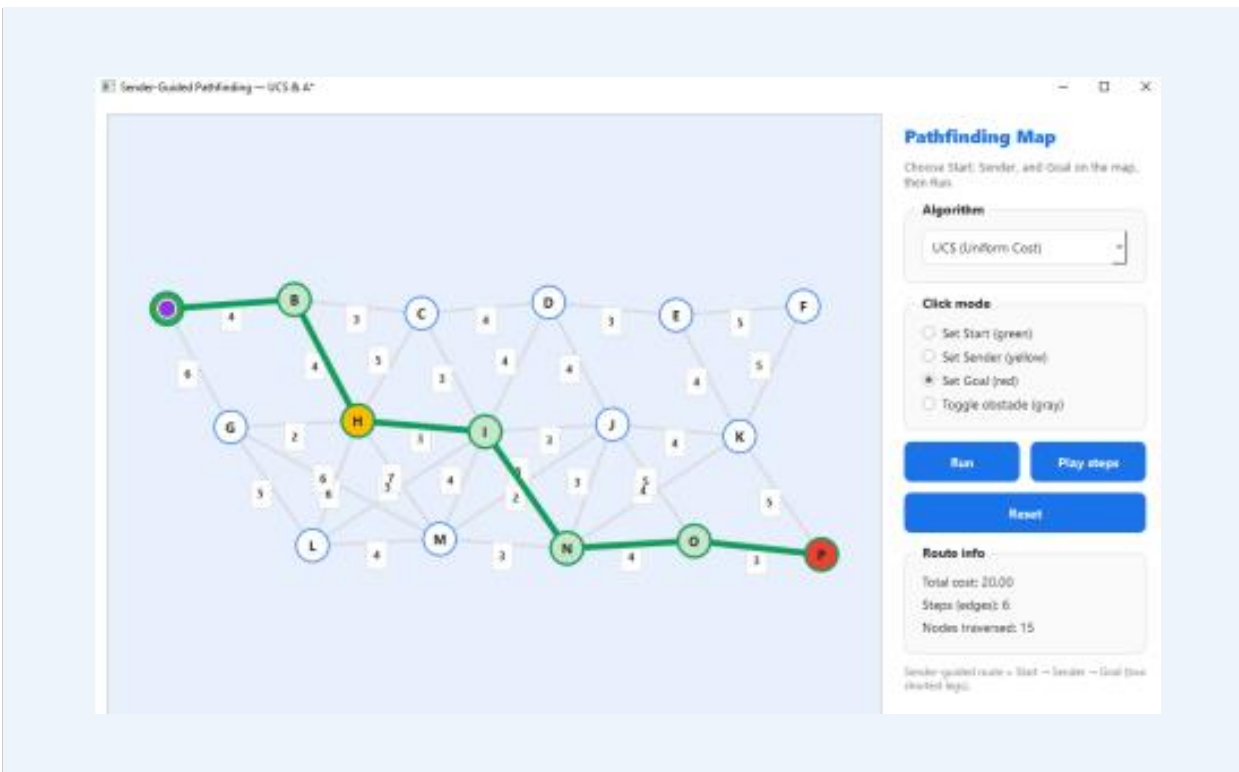
Path animation is implemented using a QTimer that fires every 420 ms (by default). On each tick, the agent QGraphicsEllipseItem is repositioned to the next node in the path sequence. The animation can be triggered independently of the Run action via the Play Steps button.

7.5 Edge Label Rendering

Edge weight labels are drawn at the midpoint of each edge with a slight perpendicular offset to avoid overlap with the edge line. A white background rectangle (QGraphicsRectItem) is placed behind each label for readability. Both label and background are set to Qt.NoButton to pass mouse events through to the underlying node items.

8. Sample Runs & Results

8.1 Run 1 — UCS: A → H → P



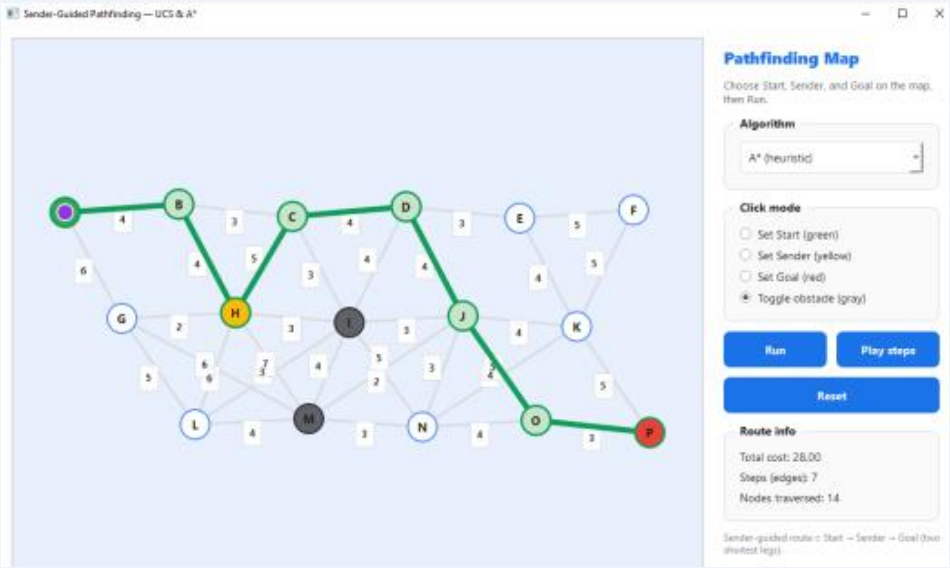
Expected path: A → B → H → I → N → O → P

Expected cost: 4 + 4 + 5 + 4 + 3 = 20.00

8.2 Run 2 — A*: A → H → P

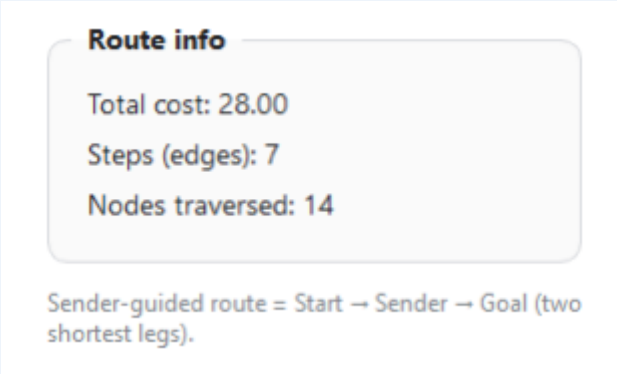
A* should produce the same optimal path as UCS for this configuration, while potentially expanding fewer nodes internally due to heuristic guidance.

8.3 Run 3 — Obstacle Test



When key intermediate nodes are blocked, both algorithms correctly find an alternate route (if one exists) or report 'No path found.'

8.4 Route Info Panel



9. Testing & Validation

9.1 Correctness

Manual verification was performed by tracing edge weights along returned paths and confirming they match the reported total cost. Both UCS and A* were tested on the same start/sender/goal triples and produced identical optimal paths, confirming that the Euclidean heuristic is admissible.

9.2 Edge Cases Tested

- Start == Goal: Not permitted — validated by GUI warning.
- Non-distinct nodes: A warning dialog prevents Start, Sender, and Goal from being the same node.
- All paths blocked: Algorithm returns None and GUI shows a 'No route exists' dialog.
- Sender blocked: search correctly returns infinity / no path.
- Single-node path: handled gracefully (no path segments drawn).

9.3 Performance

On the 16-node default graph, both algorithms run in sub-millisecond time. A* consistently expands fewer nodes than UCS due to heuristic guidance, though the difference is negligible on a graph of this size.

10. Conclusion

This project successfully demonstrates two classical AI search algorithms — UCS and A* — within an interactive, visually rich PyQt5 application. The sender-guided routing model adds a practical constraint that reflects real-world multi-stop navigation scenarios.

Key takeaways:

- UCS guarantees optimality without any domain-specific knowledge; A* achieves the same optimality with improved efficiency by leveraging the admissible Euclidean heuristic.
- The clean separation between the core graph/algorithm layer and the GUI layer makes the codebase extensible — additional algorithms (e.g., Bidirectional A*, Greedy BFS) can be added with minimal changes.
- The obstacle-blocking feature provides an intuitive way to explore how pathfinding adapts to dynamic environments.

10.1 Future Improvements

- Add Greedy Best-First Search and Bidirectional A* for comparison.
- Support custom graph creation (add/remove nodes and edges via GUI).
- Display step-by-step node expansion during search (open/closed set visualization).
- Export path and cost data to CSV or JSON.
- Add support for directed (one-way) edges.

References

- Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.
- Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1(1), 269–271.
- PyQt5 Documentation. https://doc.qt.io/qtforpython-5/
- Python Software Foundation. (2024). heapq — Heap queue algorithm. https://docs.python.org/3/library/heapq.html